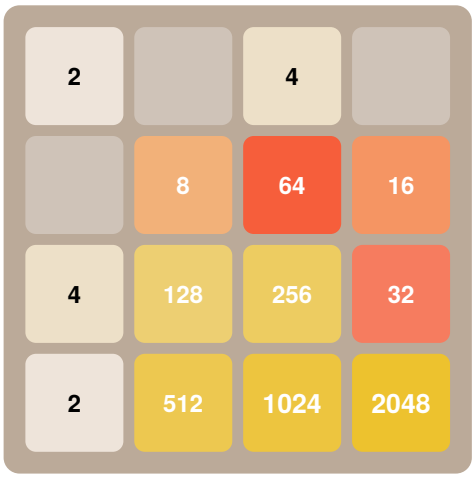
Solving 2048: Classical RL vs. LLM-Guided RLVR

CS 5180 — Reinforcement Learning — Northeastern University

Shriman Raghav & Gautham Ramkumar — MS Robotics

The 2048 Puzzle

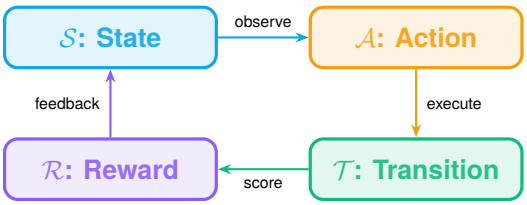


Slide tiles in 4 directions. Equal tiles merge, doubling their value. Reach **2048** to win! Random new tiles (2 or 4) spawn after each valid move.

Why 2048 Is Hard for RL

- **Sparse rewards** — hundreds of low-value merges before one big payoff
- **Stochastic dynamics** — random tile spawns prevent deterministic planning
- **Huge state space** — $\sim 17^{16} \approx 2.8 \times 10^{19}$ configurations
- **Long horizons** — episodes last 100–1000+ steps

Formal MDP



$\mathcal{S}$ 4×4 board $\rightarrow$ 16-channel binary tensor $\mathbf{x} \in \{0,1\}^{16\times4\times4}$

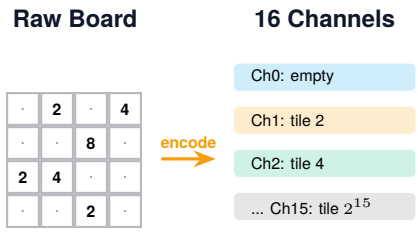
$\mathcal{A}$ {UP, RIGHT, DOWN, LEFT} — 4 discrete actions

$\mathcal{T}$ Slide-merge (deterministic) $\oplus$ random tile spawn (stochastic: 2 w/ $p{=}0.9$, 4 w/ $p{=}0.1$)

$\mathcal{R}$ Merge score delta + milestone bonuses (+50@256, +100@512, +200@1024)

γ 0.99

Observation Encoding



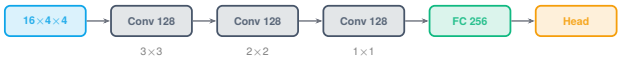
$x_{k,i,j} = \mathbb{1}[B_{i,j} = 2^k]$ — one-hot power-of-2 avoids magnitude bias in CNN filters.

7 Agents benchmarked end-to-end • **5M training steps** per agent • **DQN reaches tile 2048**

Algorithms & Architecture

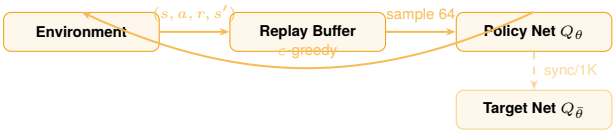
Shared CNN backbone isolates the learning algorithm — not the architecture

Shared CNN Backbone (330K params)



Three 2×2 convolutions (128 filters each) + ReLU → 256-dim features. Each algorithm adds its own task head. All deep agents share this **identical** backbone.

DQN | Best Performer ★



$\mathcal{L} = \text{SmoothL1}(Q_\theta(s,a),\ r + \gamma \max_{a'} Q_{\bar\theta}(s',a'))$

Key: Buffer 100K, ε: $1.0 \to 0.01$ over 100K steps, lr 10^{-4}, grad clip 1.0

PPO | MaskablePPO

Clipped surrogate with invalid-action masking:

$\mathcal{L} = \min\Big(r_t\hat{A}_t,\ \text{clip}(r_t, 1{\pm}0.2)\hat{A}_t\Big)$

GAE $\lambda{=}0.95$, 2048 steps/rollout, 10 epochs/batch, 8 parallel envs. Logits $\to -\infty$ for invalid actions.

A2C | MaskableA2C

$\mathcal{L} = -\mathbb{E}[\log \pi\hat{A}] + 0.5\|R{-}V\|^2 - 0.01\,\mathcal{H}[\pi]$

5-step bootstrap, entropy reg. for exploration, 8 parallel envs.

QR-DQN | Distributional RL

Learns 50 quantiles of the return distribution:

$\mathcal{L}_{\text{QR}} = \frac{1}{50}\sum_i \mathbb{E}_j[\rho^\kappa_{\tau_i}(\delta_{ij})]$

Net outputs $4{\times}50{=}200$ values. Action by mean Q.

SAC | Max-Entropy

$J(\pi) = \sum_t \mathbb{E}[r_t + \alpha\mathcal{H}[\pi(\cdot|s_t)]]$

Twin Q-nets, auto-tuned temperature α, Polyak $\tau{=}0.005$.

LFA | Only 108 Parameters!

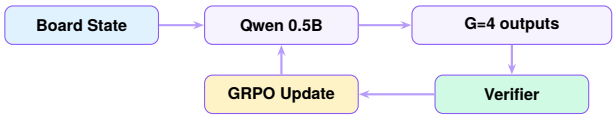
Surprise: 108 params beats 330K–494M deep nets!

$\hat{q}(s,a) = \mathbf{w}_a^\top \phi(s), \quad \mathbf{w} \in \mathbb{R}^{4\times 27}$

27-dim features: log-tile values, empty ratio, merge potential, 4 monotonicity scores, corner bonus, tile distribution stats.

Semi-gradient SARSA: $\mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha\delta\phi(s)$

GRPO | LLM + Verifiable Rewards



Model: Qwen2.5-0.5B, QLoRA 4-bit, 5.2M trainable (1.05%)

$\hat{A}_i = (r_i - \mu_G)/\sigma_G$ (group-normalized, no critic!)

Rewards: Format (+0.5) | Direction (+0.5) | Game (±2) | Quality (+0.5)

Staged scheduling required! All rewards at once → reward hacking (model maximizes length, ignores format).

Results

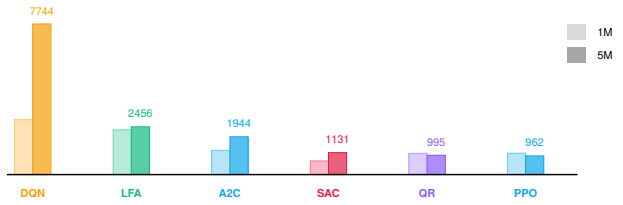
5M training steps · Shaped rewards · NVIDIA RTX 4060 GPU

Final Performance Comparison

Agent	Avg Score	Max Score	Max Tile	512+ Rate	1024+ Rate	Parameters
DQN ★	7,744	39,314	2048	76.0%	31.0%	330K
LFA	2,456	11,436	1024	6.0%	<1%	108
A2C	1,944	9,640	1024	6.0%	<1%	330K
GRPO (LLM)	1,816	1,816	128	—	—	494M
SAC*	1,131	5,566	512	<1%	—	330K
QR-DQN	995	5,176	512	<1%	—	430K
PPO	962	4,900	512	<1%	—	330K

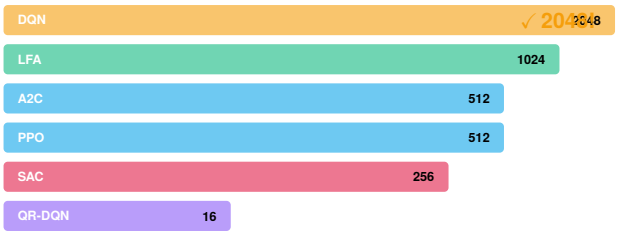
*SAC trained ∼4M steps (in progress); all others completed 5M.

Scaling: 1M → 5M Steps



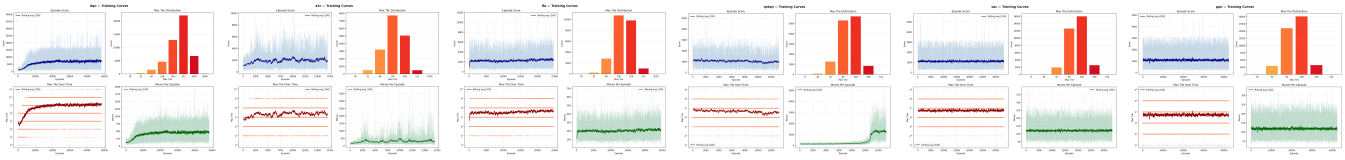
DQN: +174% scaling — PPO: −12% degradation

Hunt-2048 Mode



Only DQN reaches tile 2048 (score 21,572). Adding 5% ε-noise breaks deterministic policy cycles.

Learning Curves



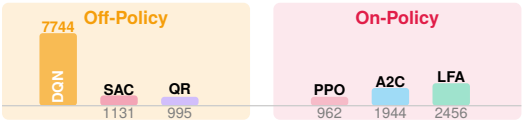
DQN | 7,744 A2C | 1,944 LFA | 2,456 QR | 995 SAC | 1,131 PPO | 962

Off-policy replay is the decisive factor. DQN's 100K buffer lets it revisit rare high-scoring trajectories: the key to conquering 2048's sparse rewards.

Analysis & Conclusions

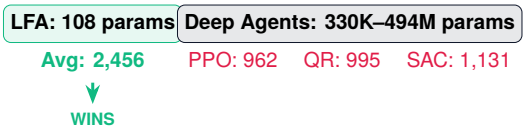
Key insights from 7 agents $\times$ 5M training steps

1 Off-Policy $\gg$ On-Policy



DQN's **experience replay** (100K buffer) revisits rare high-scoring trajectories thousands of times. On-policy methods discard each trajectory after one update, which is catastrophic for sparse rewards.

2 Feature Engineering vs. Deep Learning



Hand-crafted monotonicity & corner features provide stronger inductive bias than CNNs learning from scratch. **4000$\times$ parameter efficiency!** But LFA plateaus at $\sim$2500 (linear capacity ceiling).

3 Scaling Reveals Divergent Trajectories

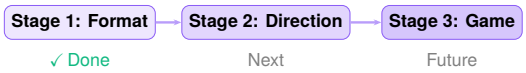
Pattern	Δ	Agents
$\uparrow$ **Strong**	**+174%**	DQN
$\uparrow$ Modest	+56–62%	A2C, SAC
$\rightarrow$ Plateau	+7%	LFA
$\downarrow$ **Degrade**	**$-$9–12%**	PPO, QR-DQN

More training doesn't always help! PPO *gets worse* with 5$\times$ more data. QR-DQN develops invalid-move loops (moves/episode: 200 $\rightarrow$ 1000+).

4 Exploration Breaks Policy Cycles

Deterministic greedy policies repeat suboptimal moves at high-tile states. Just **5% random noise** breaks cycles $\rightarrow$ DQN reaches 2048 in Hunt mode.

5 GRPO: Format Before Strategy



Simultaneous rewards $\rightarrow$ **reward hacking** (model generates filler text). Must learn XML format first, then strategy.

Inference Latency

LLM is **400$\times$ slower** but provides **interpretable** chain-of-thought reasoning.

Future Work

- Complete SAC 5M training
- Staged GRPO (Stage 2: +game reward)
- Scaling curves at checkpoints 50K$\rightarrow$5M
- Multi-seed experiments for error bars
- Hybrid LLM-MCTS exploration

Key Takeaways

1. DQN dominates (avg 7,744, tile 2048) via experience replay **2.** LFA (108 params) beats all deep on-policy agents **3.** PPO degrades with more training **4.** GRPO needs staged rewards **5.** 5% exploration noise is essential for reaching 2048